Rapport de securite - Hair Corner

Analyse statique du frontend Angular, backend FastAPI, authentification, uploads, configuration Docker et modeles de donnees. Aucun changement de code n'a ete applique pendant cette analyse.

Niveau	Nombre	Resume
Critique	2	Secrets/identifiants par defaut; uploads trop permissifs et publics.
Eleve	4	JWT dans localStorage, duree longue, absence de rate limiting, Docker dev expose.
Moyen	4	Taille upload non limitee, totaux commande manipulables, health DB public, CORS a surveiller.

Verdict global

Le site est acceptable pour un environnement local ou de demonstration, mais il ne doit pas etre expose en production sans durcissement. Les priorites sont les secrets, les uploads, la protection du token admin et l'anti-spam.

Perimetre analyse

- Frontend Angular: pages publiques, login admin, stockage token, interceptor et guard.
- Backend FastAPI: routes publiques/admin, auth JWT, Pydantic, uploads et exposition statique.
- Configuration: Docker Compose, Dockerfiles, CORS, secrets par defaut.

Constats detailles

Critique - Identifiants et secrets par defaut en dur

Fichiers: backend/app/config.py, docker-compose.yml, frontend/src/app/pages/login.page.ts

Risque: Le projet contient JWT_SECRET_KEY=change-this-secret-in-production, ADMIN_PASSWORD=admin12345, POSTGRES_PASSWORD=hair_corner_password. Le login admin est aussi pre-rempli avec admin12345. Si le deploiement utilise ces valeurs, un attaquant peut prendre le controle de l'administration.

Recommandation: Supprimer les secrets du code et de docker-compose partage, exiger des variables d'environnement fortes, retirer le mot de passe pre-rempli du formulaire login, faire tourner une rotation des secrets avant mise en production.

Critique - Uploads trop permissifs servis publiquement

Fichiers: backend/app/routes/uploads.py, backend/app/main.py

Risque: Les uploads acceptent image, video, PDF, Word, texte et application/octet-stream, puis sont exposes publiquement sous /uploads. Cela peut transformer le site en hebergement de fichiers non maitrises ou dangereux.

Recommandation: Limiter aux types strictement necessaires, idealement images uniquement; verifier extension, MIME et signature fichier; imposer une taille maximale; stocker hors webroot ou servir avec headers de securite; renommer de facon controlee.

Eleve - JWT admin stocke dans localStorage

Fichiers: frontend/src/app/shared/auth.service.ts, frontend/src/app/shared/auth.interceptor.ts

Risque: Le token admin est lisible par JavaScript. Une faille XSS future permettrait de voler directement la session admin.

Recommandation: Preferer un cookie HttpOnly, Secure, SameSite. Si localStorage est conserve temporairement, reduire fortement la duree de vie du token et renforcer les controles XSS/CSP.

Eleve - Duree JWT longue

Fichiers: backend/app/config.py

Risque: jwt_expire_minutes vaut 10080, soit 7 jours. Un token vole reste exploitable longtemps.

Recommandation: Reduire la duree de vie, par exemple 15 a 60 minutes, avec refresh token securise si necessaire.

Eleve - Absence de rate limiting

Fichiers: backend/app/routes/auth.py, messages.py, orders.py, reservations.py

Risque: Les endpoints login, messages, commandes et reservations peuvent etre spammes ou bruteforces.

Recommandation: Ajouter un rate limiting par IP et par compte sur login; ajouter CAPTCHA ou throttling doux sur formulaires publics; journaliser les echecs repetes.

Eleve - Docker Compose oriente developpement

Fichiers: docker-compose.yml

Risque: Le backend utilise --reload, monte le code source en volume, expose les ports et conserve des secrets par defaut.

Recommandation: Creer une configuration production separee: pas de --reload, pas de montage source, secrets via environnement/secret manager, reverse proxy HTTPS, ports internes non exposes inutilement.

Constats moyens

Moyen - Pas de limite de taille upload

Fichiers: backend/app/routes/uploads.py

Risque: Le code lit les fichiers par chunks mais ne limite pas la taille totale. Un admin compromis ou abus interne peut remplir le disque.

Recommandation: Fixer une taille maximale serveur et proxy, par exemple 5-10 MB pour images, et refuser au-dela.

Moyen - Totaux de commande calcules par le client

Fichiers: backend/app/models/order.py, backend/app/routes/orders.py

Risque: Le client envoie subtotal, delivery_fee et total. Le backend verifie seulement que les montants sont positifs, sans recalcul depuis un catalogue serveur.

Recommandation: Recalculer les prix et totaux cote backend a partir des produits connus; ne jamais faire confiance aux montants transmis par le navigateur.

Moyen - Endpoint sante base de donnees public

Fichiers: backend/app/main.py

Risque: /db/health revele publiquement l'etat de connexion a la base. Cela donne de l'information utile a un attaquant.

Recommandation: Restreindre cet endpoint au reseau interne, l'authentifier, ou le retirer en production.

Moyen - CORS a surveiller

Fichiers: backend/app/main.py, backend/app/config.py

Risque: La configuration actuelle limite les origines locales, mais allow_credentials=True avec allow_methods et allow_headers larges devient risque si CORS_ORIGINS est elargi.

Recommandation: Conserver une liste stricte d'origines production et eviter les wildcards, surtout avec credentials.

Points positifs observes

- Les mots de passe admin sont hashes avec bcrypt.

- Les routes admin principales passent par get_current_admin.

- SQLAlchemy est utilise; pas de concatenation SQL dangereuse observee sur les entrees utilisateur.

- Pydantic impose des longueurs sur de nombreux champs publics.

- Angular echappe le HTML par defaut; aucun usage dangereux evident de innerHTML ou DomSanitizer n'a ete observe.

Plan d'action prioritaire

- Changer tous les secrets et retirer les valeurs par defaut dangereuses.

- Supprimer le mot de passe pre-rempli du login admin.

- Durcir les uploads: type, signature, taille, stockage et headers.

- Ajouter rate limiting sur login et formulaires publics.

- Remplacer localStorage par cookies HttpOnly ou reduire fortement la duree JWT.

- Recalculer les totaux de commande cote backend.
- Creer une configuration Docker production separee.
- Restreindre ou retirer /db/health en production.

Conclusion

Hair Corner a une base technique saine pour un MVP local, mais les valeurs par defaut, les uploads publics et la gestion du token admin sont les points a corriger avant exposition publique. Une fois ces elements traites, le risque global baissera nettement.